

IoT: MQTT

Information Technology I

Carlos Rojas-Sánchez

Universidad del Mar

Contenido

1. Fundamentos de IoT
2. MQTT
3. Práctica MQTT desde Terminal
4. ESP8266: Introducción y arquitectura
5. MicroPython en ESP8266

Fundamentos de IoT

¿Qué es IoT?

El **Internet de las Cosas (IoT)** es un paradigma que permite conectar objetos físicos a una red para **capturar, intercambiar y procesar datos**.

- Objetos físicos
- Sensores y actuadores
- Redes de comunicación
- Procesamiento de datos
- Aplicaciones y servicios

De un objeto a un dispositivo IoT

Objeto físico $\longrightarrow$ Sensor $\longrightarrow$ Conectividad $\longrightarrow$ Datos

Ejemplo

Un sensor de temperatura mide la temperatura de un salón, conecta el dispositivo a una red Wi-Fi y envía el valor a un servidor.

27.4 °C

¿Qué elementos componen un sistema IoT?

1. **Dispositivo:** obtiene información del entorno.
2. **Sensor:** transforma una magnitud física en datos.
3. **Conectividad:** permite transportar los datos.
4. **Procesamiento:** analiza o transforma la información.
5. **Aplicación:** presenta la información al usuario.
6. **Actuador:** ejecuta una acción sobre el entorno.

Sensor $\rightarrow$ **Red** $\rightarrow$ **Procesamiento** $\rightarrow$ **Aplicación** $\rightarrow$ **Acción**

Sensores

Un sensor es un dispositivo que permite **detectar o medir una magnitud física** y convertirla en información que puede ser procesada.

- Temperatura
- Humedad
- Luz
- Movimiento
- Presión
- Distancia
- Sonido
- Gas
- Aceleración
- GPS

Actuadores

Un actuador es un dispositivo que recibe una señal de control y produce una **acción física**.

- LED
- Motor
- Servomotor
- Relé
- Bomba de agua
- Cerradura electrónica

Ejemplo
Si un sensor detecta una temperatura elevada, el sistema puede activar automáticamente un ventilador.

Dispositivos IoT

Los dispositivos IoT normalmente combinan:

- Microcontrolador o computadora.
- Sensores.
- Actuadores.
- Interfaz de comunicación.
- Fuente de alimentación.
- Software.

Ejemplos
ESP32, ESP8266, Arduino, Raspberry Pi, sensores inteligentes y dispositivos industriales.

Microcontroladores y IoT

Microcontrolador
Es un circuito integrado diseñado para ejecutar un programa y controlar dispositivos electrónicos.

En IoT puede encargarse de:

- Leer sensores.
- Procesar información.
- Controlar actuadores.
- Conectarse a una red.
- Enviar y recibir datos.

ESP32 + Wi-Fi + sensores = dispositivo IoT

Conectividad en IoT

Los dispositivos IoT necesitan algún mecanismo para **intercambiar información**.

Conexiones inalámbricas

- Wi-Fi
- Bluetooth
- Zigbee
- LoRa / LoRaWAN
- Redes celulares

Conexiones cableadas

- Ethernet
- RS-485
- CAN
- Modbus

El flujo de información en IoT

Sensor $\longrightarrow$ Dispositivo $\longrightarrow$ Red $\longrightarrow$ Servidor $\longrightarrow$ Aplicación

Ejemplo: temperatura

DHT22 $\rightarrow$ ESP32 $\rightarrow$ Wi-Fi $\rightarrow$ MQTT Broker $\rightarrow$ Dashboard

El dato se convierte en información útil.

Arquitectura básica de IoT

Capa física

Sensores y actuadores

↓

Capa de comunicación

Wi-Fi, Ethernet, LoRa, Bluetooth

↓

Capa de aplicación

Servidor, base de datos, dashboard

IoT: datos en tiempo real

Una característica importante de muchos sistemas IoT es la capacidad de trabajar con información **casi en tiempo real**.

- Un sensor genera un dato.
- El dispositivo lo envía.
- El servidor lo recibe.
- La aplicación lo procesa.
- El usuario observa el resultado.

$$\text{Medir} \rightarrow \text{Comunicar} \rightarrow \text{Procesar} \rightarrow \text{Actuar}$$

Estación IoT

- Temperatura
- Humedad
- Presión atmosférica
- Intensidad de luz
- Calidad del aire

Funcionamiento

Los sensores toman mediciones periódicamente y el dispositivo envía los datos mediante una red hacia un servidor donde pueden almacenarse, analizarse y visualizarse.

¿Dónde encontramos IoT?

- Hogar
- Agricultura
- Industria
- Salud
- Transporte
- Ciudades
- Energía
- Medio ambiente
- Logística
- Seguridad

IoT conecta el mundo físico con el mundo digital.

IoT vs. Internet tradicional

	Internet tradicional	IoT
Dispositivo	Computadora / teléfono	Sensor / actuador
Datos	Generados por usuarios	Generados por objetos
Interacción	Persona–sistema	Objeto–sistema
Acción	Principalmente digital	Puede ser física
Ejemplo	Página web	Sensor de temperatura

¿Por qué necesitamos protocolos?

Los dispositivos IoT necesitan reglas para comunicarse.

Un protocolo define, entre otras cosas:

- Cómo se estructura un mensaje.
- Cómo se envían los datos.
- Cómo se identifica un dispositivo.
- Cómo se detectan errores.
- Cómo se establece la comunicación.

Algunos protocolos utilizados en IoT
HTTP, MQTT, CoAP, AMQP, WebSocket y protocolos industriales.

¿Por qué MQTT es importante en IoT?

MQTT
Message Queuing Telemetry Transport es un protocolo ligero diseñado para el intercambio de mensajes, especialmente útil en dispositivos con recursos limitados.

- Bajo consumo de recursos.
- Comunicación mediante mensajes.
- Modelo **publish/subscribe**.
- Permite desacoplar dispositivos.
- Muy utilizado en sistemas IoT.

$$\text{Sensor} \rightarrow \text{MQTT Broker} \rightarrow \text{Aplicación}$$

Resumen: fundamentos de IoT

IoT = Mundo físico + Datos + Conectividad + Software

- Los **sensores** capturan información.
- Los **microcontroladores** procesan datos.
- Las **redes** permiten la comunicación.
- Los **protocolos** establecen las reglas.
- Los **servidores** almacenan y procesan información.
- Las **aplicaciones** presentan los datos.
- Los **actuadores** permiten ejecutar acciones.

MQTT

¿Qué es MQTT?

MQTT
Message Queuing Telemetry Transport es un protocolo ligero de comunicación basado en el modelo **Publish/Subscribe**.

MQTT fue diseñado especialmente para escenarios donde:

- Los dispositivos tienen recursos limitados.
- La conexión de red puede ser inestable.
- Se requiere intercambiar pequeños mensajes.
- Existen muchos dispositivos comunicándose.

MQTT es uno de los protocolos más utilizados en IoT.

Arquitectura de MQTT

Publisher $\longrightarrow$ **Broker** $\longrightarrow$ Subscriber

- El **Publisher** produce o envía información.
- El **Broker** recibe y distribuye los mensajes.
- El **Subscriber** recibe la información que le interesa.

Idea fundamental
El Publisher y el Subscriber **no necesitan conocerse directamente**.

El Broker MQTT

El broker es el componente central encargado de **recibir, filtrar y distribuir mensajes MQTT**.

Entre sus funciones se encuentran:

- Recibir mensajes de los Publishers.
- Identificar el Topic del mensaje.
- Mantener las suscripciones.
- Entregar los mensajes a los Subscribers correspondientes.
- Gestionar las conexiones de los clientes.

El Broker es el intermediario de la comunicación.

Ejemplos de Brokers MQTT

Existen diferentes implementaciones de brokers MQTT.

- **Mosquitto**
- EMQX
- HiveMQ
- VerneMQ

Para nuestro curso
Utilizaremos **Mosquitto** para crear un broker MQTT local y realizar nuestras primeras pruebas.

Cliente MQTT $\longleftrightarrow$ Mosquitto

Publisher

Es un cliente MQTT que **publica mensajes** en un Topic.

Un Publisher puede ser:

- Un ESP32.
- Un Raspberry Pi.
- Una computadora.
- Un programa en Python.
- Una aplicación Node.js.
- Un sensor IoT.

Ejemplo
Un ESP32 mide una temperatura de 27.4 °C y publica ese valor mediante MQTT.

Subscriber

Es un cliente MQTT que se **suscribe a uno o más Topics** para recibir mensajes.

Un Subscriber puede ser:

- Una computadora.
- Un ESP32.
- Un programa Python.
- Una aplicación Node.js.
- Un dashboard web.

Ejemplo
Un dashboard está suscrito al Topic `iot/temperatura` y recibe las mediciones.

Topics

Un Topic es una cadena de texto utilizada para **organizar y dirigir los mensajes MQTT**.

Los Topics pueden organizarse jerárquicamente:

`iot/temperatura`

`iot/aula1/temperatura`

`iot/aula1/humedad`

`iot/campus/laboratorio/temperatura`

Los Topics permiten organizar los datos de IoT.

¿Cómo funciona `publish`?

El Publisher envía un mensaje al Broker indicando:

1. El Topic.
2. El contenido del mensaje.
3. El nivel de QoS.

Publisher

↓

Broker

Ejemplo
Topic: `iot/aula1/temperatura` Mensaje: `27.4`

¿Cómo funciona `subscribe`?

El Subscriber indica al Broker qué Topics desea recibir.

Subscriber

$\downarrow$

Broker

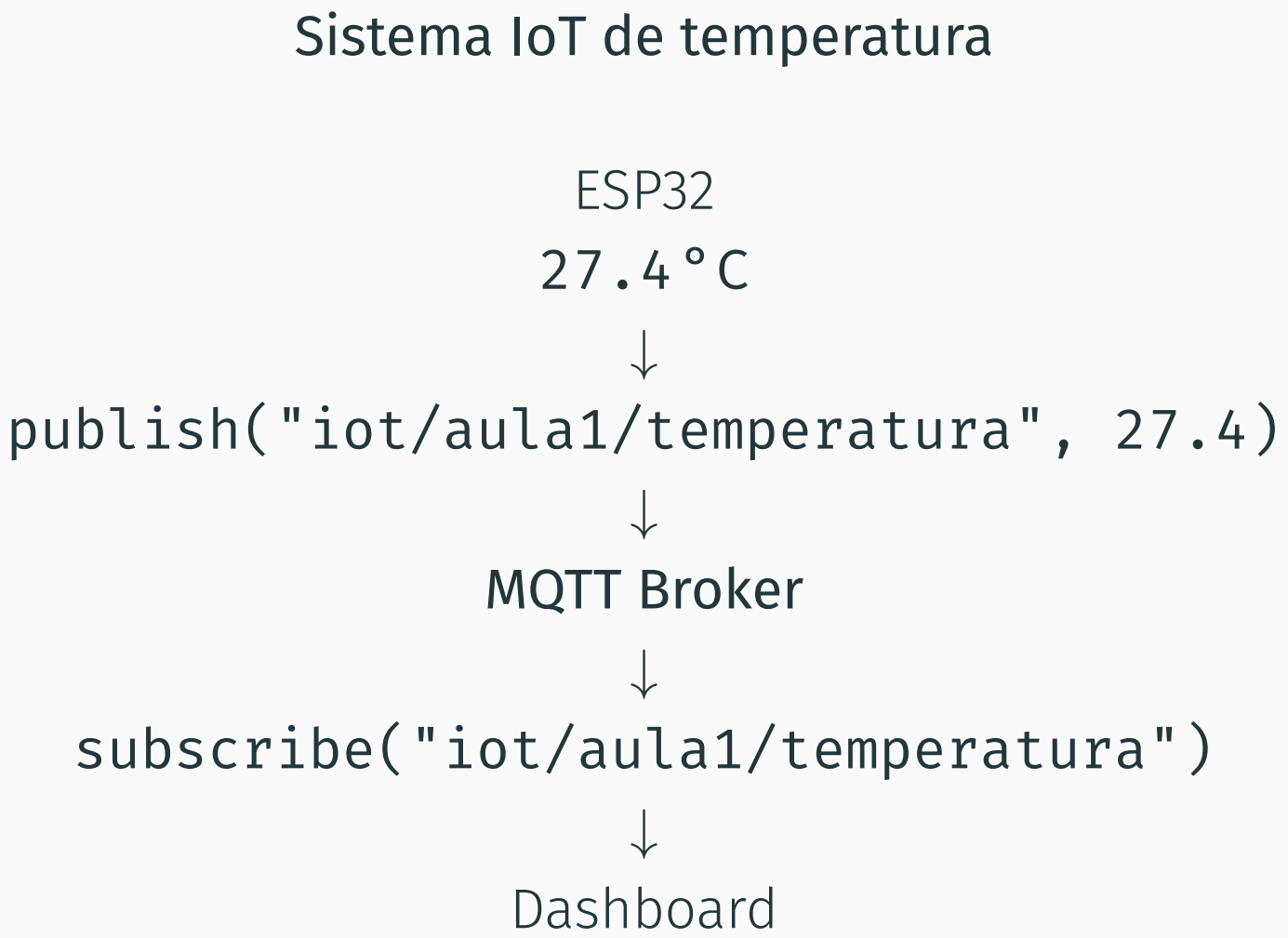
Después:

$$\text{Publisher} \rightarrow \text{Broker} \rightarrow \text{Subscriber}$$

Importante
El Subscriber no necesita conocer la dirección del Publisher.

Solamente necesita conectarse al Broker y suscribirse al Topic.

Ejemplo completo: temperatura



QoS: Quality of Service

Quality of Service determina el nivel de garantía con el que MQTT entrega un mensaje.

MQTT define tres niveles:

- **QoS 0** – At most once
- **QoS 1** – At least once
- **QoS 2** – Exactly once

Mayor QoS $\Rightarrow$ mayor garantía y mayor overhead de comunicación

Comparación de QoS

QoS	Entrega	Duplicados	Sobrecarga
0	Como máximo una vez	No	Baja
1	Al menos una vez	Posibles	Media
2	Exactamente una vez	No	Alta

Regla práctica
No siempre necesitamos QoS 2. El nivel adecuado depende de la importancia del dato y de los recursos disponibles.

Resumen de MQTT

- **Broker:** intermediario de los mensajes.
- **Publisher:** publica mensajes.
- **Subscriber:** recibe mensajes.
- **Topic:** identifica y organiza los mensajes.
- **publish:** operación para enviar un mensaje.
- **subscribe:** operación para recibir mensajes.
- **QoS:** define la garantía de entrega.

Práctica MQTT desde Terminal

Práctica: nuestro primer sistema MQTT

Objetivo
Crear un sistema MQTT básico utilizando únicamente la terminal.

Al finalizar podremos:

- Iniciar un Broker MQTT.
- Conectar un Publisher.
- Conectar un Subscriber.
- Crear y utilizar Topics.
- Publicar mensajes.
- Recibir mensajes.
- Experimentar con diferentes niveles de QoS.

Arquitectura de la práctica

Terminal 1

`mosquitto`

↑

Broker

↑

Terminal 2 Terminal 3
Publisher Subscriber

`iot/aula/temperatura`

Paso 1: verificar Mosquitto

Primero verificaremos que Mosquitto esté instalado.

Terminal
```
mosquitto --version
```

Deberíamos obtener información similar a:

```
mosquitto version 2.x.x
```

Si el comando no existe, debemos instalar Mosquitto y sus clientes MQTT.

Paso 2: iniciar el Broker

Abrimos una terminal y ejecutamos:

```
mosquitto -v
```

La opción:

- `-v` muestra información detallada de las conexiones.

Importante
No cerremos esta terminal. Aquí estará ejecutándose nuestro Broker MQTT.

¿Qué está haciendo Mosquitto?

En este momento tenemos:

Mosquitto

↓

Broker MQTT

↓

Esperando clientes

El Broker normalmente escucha conexiones MQTT en el puerto: **1883**

Paso 3: crear un Subscriber

Abrimos una **segunda terminal**.

Ejecutamos:

```
mosquitto_sub -h localhost -t iot/aula/temperatura
```

Significado:

- `mosquitto_sub`: cliente Subscriber.
- `-h localhost`: Broker local.
- `-t`: especifica el Topic.
- `iot/aula/temperatura`: Topic.

¿Qué sucede?
La terminal queda esperando mensajes.

Paso 4: crear un Publisher

Abrimos una **tercera terminal**.

Ejecutamos:

```
mosquitto_pub -h localhost \
-t iot/aula/temperatura \
-m "27.4"
```

Significado:

- `mosquitto_pub`: cliente Publisher.
- `-h`: dirección del Broker.
- `-t`: Topic.
- `-m`: mensaje.

Paso 5: observar el resultado

Al ejecutar el Publisher:

```
27.4
```

el Subscriber debería mostrar:

```
27.4
```

Publisher $\longrightarrow$ Broker $\longrightarrow$ Subscriber

¡Primera comunicación MQTT!
El mensaje viajó desde un cliente hasta otro utilizando el Broker.

Paso 6: publicar varios mensajes

Podemos publicar diferentes valores:

```
mosquitto_pub -h localhost \
-t iot/aula/temperatura -m "26.8"

mosquitto_pub -h localhost \
-t iot/aula/temperatura -m "27.1"

mosquitto_pub -h localhost \
-t iot/aula/temperatura -m "27.9"
```

El Subscriber recibirá:

```
26.8
27.1
27.9
```

Paso 7: cambiar de Topic

Ahora publicaremos en otro Topic:

```
mosquitto_pub -h localhost \
-t iot/aula/humedad \
-m "68"
```

¿Qué ocurre?

Si nuestro Subscriber está escuchando:

```
iot/aula/temperatura
```

Resultado
No recibirá el mensaje de humedad.

El Topic determina qué mensajes recibe cada Subscriber.

Paso 8: suscribirse a otro Topic

Podemos abrir otra terminal y ejecutar:

```
mosquitto_sub -h localhost \
-t iot/aula/humedad
```

Ahora tenemos:

Subscriber 1	Subscriber 2
`temperatura`	`humedad`

Cada Subscriber recibe solamente los mensajes correspondientes al Topic al que está suscrito.

Paso 9: un Subscriber, varios Topics

Un mismo cliente puede suscribirse a varios Topics. Por ejemplo:

```
mosquitto_sub -h localhost \
-t iot/aula/temperatura \
-t iot/aula/humedad
```

Ahora el Subscriber puede recibir:

```
27.4
68
27.8
70
```

Idea importante
Un dispositivo puede publicar y suscribirse a múltiples Topics.

Paso 10: utilizar comodines

MQTT permite utilizar comodines para suscribirse a múltiples Topics. Por ejemplo:

```
mosquitto_sub -h localhost -t "iot/aula/#"
```

El símbolo # significa:

Todos los niveles posteriores

Por lo tanto recibiremos mensajes de:

```
iot/aula/temperatura
iot/aula/humedad
iot/aula/luz
iot/aula/presencia
```

Paso 11: probar QoS 0

Podemos especificar el nivel de QoS al publicar:

```
mosquitto_pub -h localhost \
-t iot/aula/temperatura -m "27.4"  -q 0
```

`-q 0` significa:

QoS 0

El mensaje se entrega como máximo una vez.

Paso 12: probar QoS 1

Ahora utilizamos:

```
mosquitto_pub -h localhost \
-t iot/aula/temperatura \
-m "28.1" \
-q 1
```

`-q 1` significa:

QoS 1

El protocolo utiliza confirmación para garantizar que el mensaje llegue al menos una vez.

Paso 13: probar QoS 2

Finalmente:

```
mosquitto_pub -h localhost \
-t iot/aula/temperatura \
-m "28.5"  -q 2
```

`-q 2` significa:

QoS 2

El protocolo utiliza un intercambio adicional de mensajes para garantizar la entrega exactamente una vez.

Práctica: reto 1

Reto
Crear un sistema MQTT que simule una estación meteorológica.

Crear los siguientes Topics:

- `iot/estacion/temperatura`
- `iot/estacion/humedad`
- `iot/estacion/luz`

Crear un Subscriber que reciba información de los tres Topics.

No utilizar ESP32 todavía.

La estación será simulada completamente desde la terminal.

Reto 1: solución esperada

Subscriber:

```
mosquitto_sub -h localhost \
-t "iot/estacion/#"
```

Publisher de temperatura:

```
mosquitto_pub -h localhost \
-t iot/estacion/temperatura -m "27.4"
```

Publisher de humedad:

```
mosquitto_pub -h localhost \
-t iot/estacion/humedad -m "72"
```

Publisher de luz:

```
mosquitto_pub -h localhost \
-t iot/estacion/luz -m "850"
```

Práctica: reto 2

Situación
Un aula tiene sensores de temperatura, humedad y presencia.

Diseñar una estructura de Topics:

`iot/`

↓

`aula1/`

↓

temperatura humedad presencia

Pregunta: ¿Cómo crearías un único Subscriber que reciba todos los datos del aula?

Reto final: diseñar la comunicación

Antes de conectar un ESP32 debemos ser capaces de diseñar nuestra comunicación MQTT.

1. Identificar los dispositivos.
2. Definir los Topics.
3. Determinar quién publica.
4. Determinar quién se suscribe.
5. Elegir el QoS.
6. Probar la comunicación desde la terminal.

Siguiente paso

Reemplazar nuestro Publisher de terminal por un **ESP32** y enviar datos reales mediante MQTT.

Resumen de la práctica

Lo que acabamos de construir

`mosquitto`

↓

Broker

↑

`mosquitto_pub`

- Broker MQTT funcionando.
- Publisher desde terminal.
- Subscriber desde terminal.
- Topics.

`mosquitto_sub`

- Publish / Subscribe.
- QoS.
- Comodines.

ESP8266: Introducción y arquitectura

¿Qué es el ESP8266?

- Microcontrolador de bajo costo.
- Integra conectividad Wi-Fi.
- Puede ejecutar programas de manera independiente.
- Permite conectar sensores y actuadores a una red.
- Es muy utilizado en proyectos de IoT.

ESP8266 = Microcontrolador + Wi-Fi

¿Por qué es importante en IoT?

El ESP8266 permite convertir un dispositivo físico en un dispositivo conectado.

Mundo físico

↓

Sensores

↓

ESP8266

↓

Wi-Fi / MQTT

↓

Internet / Red local

ESP8266 como dispositivo IoT

Un ESP8266 puede realizar tres tareas fundamentales:

1. **Capturar datos**
   Temperatura, humedad, luz, movimiento, etc.
2. **Procesar datos**
   Realizar cálculos y tomar decisiones.
3. **Comunicar datos**
   Enviar información mediante Wi-Fi y protocolos como MQTT.

Elementos principales del ESP8266

- CPU / microcontrolador
- Memoria
- GPIO
- Wi-Fi
- Interfaces de comunicación
- Alimentación eléctrica

GPIO = General Purpose Input/Output

GPIO: conectar el mundo físico

Los pines GPIO permiten conectar componentes externos.

- **Entrada:** leer sensores.
- **Salida:** controlar actuadores.

$$\text{Sensor} \rightarrow \text{GPIO} \rightarrow \text{ESP8266}$$

$$\text{ESP8266} \rightarrow \text{GPIO} \rightarrow \text{LED}$$

Ejemplo: sensor de temperatura

El ESP8266 puede recibir información de un sensor.

Sensor

↓

Temperatura: 27.4 °C

↓

ESP8266

↓

MQTT

ESP8266 y Wi-Fi

El ESP8266 puede conectarse a una red inalámbrica.

ESP8266

↓

Wi-Fi

↓

Router / Access Point

↓

Red local

Esto permite que el dispositivo pueda comunicarse con otros equipos de la red.

ESP8266 y MQTT

El ESP8266 puede utilizar MQTT para intercambiar información.

ESP8266

$\xrightarrow{\text{publish}}$

Broker MQTT

$\xrightarrow{\text{publish}}$

Clientes

Por ejemplo:

`iot/aula/temperatura`

`27.4`

Arquitectura básica IoT

La arquitectura que utilizaremos será:

Sensor

↓

ESP8266

↓

Wi-Fi

↓

MQTT Broker

↓

Aplicación

Arquitectura completa del laboratorio

Dispositivo IoT

ESP8266

↓

Wi-Fi

↓

Mosquitto

Broker MQTT

↓

Terminal Python Dashboard

¿Qué hace cada componente?

ESP8266	Captura y envía información.
Wi-Fi	Proporciona conectividad.
MQTT	Define cómo se intercambian los mensajes.
Broker	Recibe y distribuye los mensajes.
Python	Puede procesar y analizar los datos.
Dashboard	Permite visualizar la información.

Ejemplo completo

Un sensor mide:

$$27.4\,°C$$

El ESP8266 publica:

`iot/aula/temperatura`

`27.4`

El broker recibe el mensaje y lo distribuye a los clientes suscritos.

ESP8266 como Publisher

El ESP8266 puede actuar como **Publisher**.

ESP8266

$\xrightarrow{\text{PUBLISH}}$

`iot/aula/temperatura`

$\rightarrow$

`27.4`

El dispositivo produce información.

ESP8266 como Subscriber

El ESP8266 también puede recibir información.

MQTT Broker

$\xrightarrow{\text{mensaje}}$

ESP8266

Por ejemplo, recibir una orden:

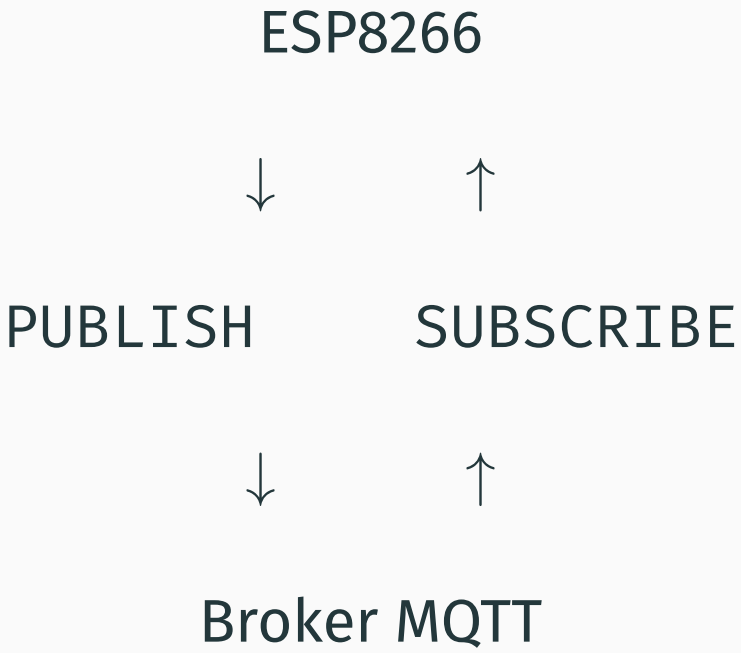
`iot/aula/led`

`ON`

El ESP8266 podría encender un LED.

Publisher + Subscriber

Un dispositivo IoT puede hacer ambas funciones.



Arquitectura del proyecto

Nuestro proyecto evolucionará progresivamente:

1. ESP8266
2. Conexión Wi-Fi
3. MicroPython
4. MQTT
5. Sensor
6. Python
7. Dashboard

Primera práctica con ESP8266

Objetivo

Conseguir que el ESP8266:

1. Ejecute MicroPython.
2. Se conecte a la red Wi-Fi.
3. Obtenga una dirección IP.
4. Se conecte al broker MQTT.
5. Publique un mensaje.

ESP8266 → Wi-Fi → MQTT

¿Qué aprenderemos después?

- Instalar MicroPython.
- Conectarnos al ESP8266.
- Configurar Wi-Fi.
- Instalar una biblioteca MQTT.
- Publicar mensajes.
- Suscribirnos a Topics.
- Conectar sensores.

Del simulador al dispositivo físico

Resumen

- El ESP8266 es un microcontrolador con Wi-Fi.
- Puede interactuar con sensores y actuadores.
- Puede funcionar como Publisher y Subscriber.
- MQTT permite comunicarlo con otros dispositivos.
- El broker centraliza la distribución de mensajes.

ESP8266 + Wi-Fi + MQTT = dispositivo IoT

MicroPython en ESP8266

ESP8266: memoria flash de 4 MB

- El NodeMCU ESP8266 de esta práctica incorpora **4 MB de memoria flash**.
- La flash conserva la información aunque se desconecte la alimentación.
- No toda la flash queda disponible para nuestros archivos: también se utiliza para el firmware y otras áreas reservadas.
- MicroPython incluye el intérprete y módulos integrados; nuestros programas se guardan en el sistema de archivos de la placa.

Idea clave
Los 4 MB son la capacidad física de la flash, no el espacio libre para nuestros programas.

¿Cómo se distribuye la memoria?



- **Firmware:** MicroPython y componentes necesarios para ejecutarse.
- **Otras áreas:** distribución de particiones y datos reservados, según el firmware.
- **Sistema de archivos:** scripts `.py`, bibliotecas y archivos de configuración.

El esquema es conceptual, no está a escala. La distribución exacta depende del firmware y de sus particiones.

Medición real en nuestra placa

Ejecutamos en la consola de Thonny:

Consultar el sistema de archivos

```
import os
print(os.statvfs("/"))
```

Resultado obtenido:

$$(4096,\ 4096,\ 763,\ 760,\ 760,\ 0,\ 0,\ 0,\ 0,\ 255)$$

- Tamaño de bloque: **4096 bytes**.
- Bloques totales: **763**.
- Bloques libres: **760**.
- Bloques ocupados: $763 - 760 = 3$.

¿Cuánto espacio queda para los archivos?

Sistema de archivos	Espacio libre
3 125 248 B	3 112 960 B
Espacio total: aproximadamente 2.98 MiB	Aproximadamente 2.97 MiB libres

- Espacio ocupado: $3 \times 4096 = 12\,288$ bytes, es decir, 12 KiB.
- Queda libre cerca del **99.6 %** del sistema de archivos.
- Estos valores describen el sistema de archivos consultado, no toda la flash física.

Ejemplo: espacio de nuestros programas

Tamaños ilustrativos de archivos para una práctica de IoT:

Archivo	Tamaño estimado
`main.py`	2 KB
`wifi.py`	1 KB
`mqtt.py`	5 KB
`config.py`	1 KB
Biblioteca MQTT	10 KB
Total	**19 KB**

Resultado

Un conjunto de archivos de unos 19 KB ocuparía menos del 1 % de los aproximadamente 2.97 MiB libres.

Los tamaños son ejemplos; pueden variar según el código y las bibliotecas utilizadas.

Flash y RAM no son lo mismo

Memoria flash

- Guarda los archivos.
- Conserva los datos al apagar.
- Capacidad física de la placa: 4 MB.

Memoria RAM

- Se utiliza mientras el programa se ejecuta.
- Guarda variables y objetos en uso.
- Es mucho más limitada que la flash.

Para consultar la RAM libre en MicroPython:

```
import gc
gc.collect()
print(gc.mem_free(), "bytes de RAM libres")
```

Conclusiones

- Los 4 MB corresponden a la memoria flash física; el firmware y las particiones ocupan parte de ella.
- En nuestra placa, `os.statvfs("/")` reporta aproximadamente **2.97 MiB libres** para archivos.
- Podemos guardar varios scripts Python, bibliotecas y archivos de configuración, siempre que quepan en el sistema de archivos.
- El tamaño del archivo no es la única limitación: también debemos vigilar la RAM disponible al ejecutar el programa.